

NETPAY

Production Readiness & Architecture Gate

Evaluation Report against Gate v1.0 (September 2026)

Repository: github.com/NetHub-Ltd/NetPay

Evaluated commit (main): 709ceb9 • Report date: 2026-09-12

RELEASE RECOMMENDATION

NO-GO

Critical gates fail: no ledger/reconciliation, no formal idempotency, incomplete payment state machine, absent payouts/refunds, and testing does not prove financial integrity. Happy-path STK collection works; that is not sufficient for production release under this gate.

Scope: application domain on **main** @ 709ceb9. Open PRs (#7 foundation, #8 CI) improve process and docs but do not close Critical financial-integrity gaps.

1. Executive Summary

NetPay is a functional multi-tenant M-Pesa orchestration service (FastAPI + React dashboard) that can create payment intents, call Daraja STK Push, ingest callback envelopes via an internal API key, update intent status, and fan out HMAC-signed merchant webhooks. That demonstrates a viable prototype and a useful internal gateway.

Against the Production Readiness & Architecture Gate, NetPay is still a **provider-centric integration layer**, not a complete payment infrastructure domain. The gate requires a coherent financial state model, deterministic idempotency, ledger truth, reconciliation, proven recovery paths, and tests that treat financial correctness as product requirements. Those are largely absent or only partially sketched.

Decision under section 1 of the gate: NO-GO.

Unresolved Critical issues include: (1) no ledger or reconciliation as source of truth; (2) no client idempotency keys / unique financial-operation constraints under concurrency; (3) payment status is a free-form string with only a partial terminal-state guard, not a tested state machine; (4) no refunds, reversals, or B2C payouts; (5) test suite does not exercise STK success/failure, duplicate callbacks, or tenant isolation races.

2. Release Scorecard (0–3 scale)

Scale per gate: 0 = absent/unsafe; 1 = implemented but not proven; 2 = implemented and tested; 3 = production-proven, observable, operationally owned. Highlighted rows are Critical with score below 2.

Core payment domain	1/3	Critical	UUID intents + tenant + provider refs exist; no formal state machine, no idempotency key, Decimal not minor units.
Collections	1/3	Critical	M-Pesa STK end-to-end path exists; C2B is URL registration only; no cards/PayPal; not provider-neutral API.
Payouts / refunds / reversals	0/3	Critical	No domain objects, APIs, or tests.
Ledger / reconciliation	0/3	Critical	No ledger, settlement compare, exception queue, or merchant statement.
Webhooks / async	1/3	Critical	Outbound HMAC webhooks + inbound envelope path; terminal duplicate short-circuit; no durable queue/DLQ.
Security / tenant isolation	1/3	Critical	JWT + can_access_tenant + internal key; credentials in DB; no rate limits; AS auth not yet wired.
API / developer experience	1/3	High	Useful REST surface + OpenAPI via FastAPI; weak idempotency docs; sandbox simulate only.
Reliability / failure handling	1/3	Critical	Some settled-state ignore on callback; no timeout/unknown semantics tests; no crash/replay durability tests.
Observability / operations	1/3	High	Loguru structured logs + /health; no metrics, alerts, or financial runbooks.
Testing / CI	1/3	Critical	~6 acceptance tests (health/login/internal key/JWT); CI present; no financial path or concurrency tests.

Average (Critical areas only): approximately **0.7 / 3**. Gate requires Critical domains at production-credible maturity before GO.

3. Architecture Fit vs Target

Target architecture: NetPay API to Payment Domain (orders, state, ledger, idempotency, refunds, payouts, webhooks) to Provider Adapters (M-Pesa, cards, PayPal). Provider callbacks are inputs to a state machine; money state is persisted before it is authoritative.

What NetPay is today:

- API + dashboard over a thin domain centered on PaymentIntent and M-Pesa helper functions.
- Provider logic lives mainly in providers/mpesa.py and payment routes; adapter boundary is informal.
- Callbacks arrive as envelopes on POST /internal/events (expects external Worker); processor updates intent rows and fans out webhooks.
- No Orders, Ledger, PaymentAttempt, Refund, Payout, or Reconciliation entities in models or migrations.

Merchants still need awareness of M-Pesa concepts (shortcode, STK callback URLs, CheckoutRequestID). The product is closer to a multi-tenant Daraja orchestrator than a provider-neutral payment infrastructure layer.

4. Canonical Payment Model — Gate section 3

Payment identity	Stable NetPay ID + tenant + provider refs	UUID PK; tenant_id; provider_checkout_id indexed (not unique)	PARTIAL
State machine	Explicit states + legal transitions	Free-form status string; terminal skip if succeeded/failed	FAIL
Idempotency	Same key cannot duplicate money movement	No idempotency_key; no concurrent tests	FAIL
Amounts	Integer minor units; no float money	Decimal(18,2); STK uses str(int(amount))	FAIL
Audit trail	Who/what/when/provider action immutable	GatewayEvent records; mutable intent is status source	PARTIAL
Correlation	Internal ID maps to provider IDs	Checkout/merchant/receipt fields stored	PARTIAL
Tenant isolation	Every object scoped + authorized	can_access_tenant; limited negative tests	PARTIAL

Findings

Identity: PaymentIntent.id is a stable NetPay identifier with tenant and integration ownership. provider_checkout_id is indexed but not UNIQUE.

State: Statuses include created, provider_requested, succeeded, failed. No enumerated state type, transition table, or tests for illegal transitions.

Idempotency: Critical gap. Clients cannot supply an Idempotency-Key. Two creates yield two STK pushes. Envelope processing short-circuits already-settled intents — necessary but not sufficient.

Money: Gate requires integer minor units. Schema uses Decimal(18,2). Does not meet the gate standard.

5. Collections — Gate section 4

M-Pesa STK Push	Initiate, callback, reconcile, timeout/retry	Real STK; envelope match; simulate; weak timeout model	PARTIAL
M-Pesa C2B	Receive/route payment notifications	register-urls only; no full C2B domain handling	FAIL
Card / PayPal	Provider abstraction + webhooks	Not present	FAIL
Unified API	One canonical collection API	M-Pesa-shaped payment-intents API only	FAIL

STK is the strongest path: create intent, credentials, OAuth (Redis), STK Push, CheckoutRequestID, envelope, status update, webhook fan-out. Missing formal timeout/unknown handling, reconciliation, concurrency-safe initiation.

6. Payouts, Refunds and Reversals — Gate section 5

Score 0/3. No B2C, refund, or reversal models, routes, services, or tests. Critical gap for full lifecycle; collections-only release still needs a refund/reversal path for financial integrity after successful collection.

7. Ledger and Reconciliation — Gate section 6 (Critical)

Score 0/3 — primary NO-GO driver. The gate states the ledger is the financial source of truth. NetPay treats `PaymentIntent.status` as working truth. No immutable ledger, reconciliation job, exception queue, or merchant statement. `GatewayEvent` is operational audit, not a balanced financial ledger.

Required before production money movement at scale:

- Append-only ledger entries tied to payment IDs with invariants.
- Periodic provider reconciliation with explicit exception records.
- Operator runbook for unmatched provider vs NetPay rows.
- Ability to explain a disputed payment from ledger + provider references + audit.

8. Webhooks and Asynchronous Processing — Gate section 7

Authenticity (outbound)	Signed merchant webhooks	HMAC-SHA256; HTTPS-only + liveness probe	PASS-ish
Authenticity (inbound)	Provider webhook verification	Internal API key; not Daraja signature in-app	PARTIAL
Fast ack / durability	Persist before heavy work	Synchronous in request path; no queue	FAIL
Idempotent consumer	Replay yields one outcome	Terminal short-circuit; not concurrency-proven	PARTIAL
Out-of-order	Late events cannot regress state	Minimal terminal guard only	FAIL
DLQ / retry	Poison events surfaced	Not implemented	FAIL

Outbound merchant webhooks are well designed (secret, probe, delivery logs). Inbound depends on external Worker and static key. Processing is inline without durable event store, lease, or DLQ.

9. Security and Tenant Isolation — Gate section 8

Authentication: Local JWT/passwords are transitional; target is NetHub AS. Production must not rely on local passwords or default secrets.

Authorization: `require_admin` and `can_access_tenant` exist. Systematic negative tests for cross-tenant access are missing from CI.

Secrets: Provider credentials stored as plain strings in DB. App secrets via env are fine in principle; code defaults are unsafe if unchanged.

Missing: rate limiting, abuse controls, public API replay protection, PII review, proven log redaction.

10. API and Developer Experience — Gate section 9

FastAPI OpenAPI is available. /v1 resources are consistent. Gaps: financial POST idempotency docs; provider-neutral status model; sandbox matrix for timeout/duplicate/concurrent cases; deeper lifecycle documentation.

11. Reliability and Failure Engineering — Gate section 10

Positive: settled intents ignore further callbacks; STK failures store reason; Redis optional outside production. Negative: no formal unknown-after-timeout state; no bounded retry tests; no worker-crash recovery (no in-repo worker); no race tests for double create or double envelope.

12. Observability and Operations — Gate section 11

Structured Loguru logging and /health exist. k3s manifest is minimal with probe paths. Absent: metrics, alerts, financial runbooks, game-day reconstruction procedure.

13. Testing Gate — Gate section 12 (Tests Are Product)

Tests are part of the product. Current suite is about 6 acceptance tests: health, admin login, internal key reject/accept, JWT required on protected routes.

Not present: state-transition unit tests; concurrent idempotency; STK success/failure integration; duplicate callback exactly-once; cross-tenant denial; webhook signature tests; ledger/refund tests; migration safety; contract tests; failure injection.

CI runs pytest and frontend lint/build. That is infrastructure only. **Testing score: 1/3 Critical — NO-GO contributor.**

14. Minimum Release Scenarios — Gate section 13

1. **STK success + callback twice.** Partial terminal guard; not proven by test or uniqueness under race.
2. **Timeout then late success.** No explicit timeout/unknown model; not proven.
3. **Concurrent same idempotency key.** No idempotency key; FAIL.
4. **Two tenants same external reference.** Tenant checks exist; not systematically tested.
5. **B2C payout retry.** Feature absent; FAIL.
6. **Callback before API finishes.** Not designed/tested.
7. **Reversal/refund after success.** Feature absent; FAIL.
- 8-9. **Reconciliation exceptions.** No reconciliation; FAIL.
10. **Worker crash after webhook.** No durable queue worker; FAIL.
11. **Provider outage 30m.** Incomplete; not proven.
12. **Reconstruct disputed payment.** Events help; no ledger/runbook; PARTIAL.

15. What Is Already Solid

- Multi-tenant model with route-level tenant checks.
- Working STK initiation with Redis-cached OAuth tokens.
- Envelope ingest with internal API key and CheckoutRequestID matching.
- Outbound webhooks: HTTPS, liveness probe, HMAC, delivery rows, max 3/tenant.
- SPA dashboard; Docker multi-arch foundation; /health probes.
- Soft-delete mixin; Alembic schema; admin bootstrap; dual DB URL handling.
- Product direction: end-user auth via NetHub AS — correct gateway boundary.

16. What You Must Do Before Production

P0 — Financial integrity (blocking GO)

1. Idempotency-Key on financial POSTs; unique per tenant; concurrent tests.
2. Enumerated state machine with illegal-transition rejection and tests.
3. Append-only ledger; compensating entries later for refunds.
4. Integer minor-unit amounts with explicit currency.
5. Automated tests for scenarios 1–4 and 10; CI fails if red.
6. Unique provider reference constraints where one provider id maps to one intent.

P1 — Collections and async durability

7. Durable inbound event store + async processor with retry/DLQ.
8. Explicit timeout/unknown states and safe late-callback handling.
9. C2B notification mapping if in first Kenyan release scope.
10. Reconciliation job + exception queue.

P2 — Security and ops

11. NetHub AS token validation; no production local passwords.
12. Secrets manager / encryption at rest for provider credentials.
13. Rate limits; log redaction audit.
14. Metrics, alerts, runbooks for stuck payments and provider outages.
15. Production k3s manifests (Ingress, secrets, HPA, correct image refs).

P3 — Expand rails after core is trustworthy

16. Refunds/reversals with ledger compensating entries and tests.
17. B2C payouts with separate permissions, limits, idempotency.
18. Additional providers behind the same canonical Payment API.

17. Suggested Phased Path to GO

Phase A — Collections-only pilot: P0 items 1–6 for STK; prove scenarios 1–4 and 10 in CI; AS auth documented; do not market as full platform. Still Conditional until ledger/idemp exist.

Phase B — Financial truth: Ledger, reconciliation, exception ops, runbooks, metrics/alerts.

Phase C — Lifecycle: Refunds then B2C; scenarios 5–9; harden k3s.

Only after Critical scorecard items reach at least 2 with evidence should leadership consider CONDITIONAL GO or GO.

18. Architecture Review Questions — Current Answers

Q: Where is the single source of truth for payment state?

A: De facto PaymentIntent.status plus provider ids. Gate requires ledger and state machine — not met.

Q: Can the same request/callback/payout be safely delivered twice?

A: Callback: partial. Request: no idempotency key. Payout: N/A.

Q: Provider says unknown after timeout?

A: Not modeled with tests.

Q: Can Merchant A callback mutate Merchant B?

A: Matched via CheckoutRequestID to intent; inbound trusts internal key — Worker isolation must be guaranteed.

Q: Payment successful while API returns error?

A: Possible; needs client guidance and reconciliation.

Q: Provider status vs NetPay status?

A: Mixed into one status field.

Q: Provider txn NetPay does not know?

A: No reconciliation exception path.

Q: Disputed payment proof?

A: Intent + events help; no ledger statement.

Q: Replace Safaricom without changing merchant contract?

A: Not yet; API is M-Pesa-shaped.

Q: Worker crash mid-lifecycle?

A: No durable queue in-repo; recovery not proven.

19. Formal Gate Sign-Off (Evaluation Draft)

Release recommendation: NO-GO

Critical outstanding issues:

1. No ledger / reconciliation as financial source of truth (Gate section 6).
2. No client idempotency keys or concurrency-proven single financial effect (Gate sections 3 and 13).
3. Payment state machine and amount model do not meet gate; refunds/payouts absent (Gate sections 3 and 5).
4. Test suite does not prove financial scenarios; tests not yet treated as product (Gate sections 12–13).
5. Provider credential storage and NetHub AS production auth boundary incomplete (Gate section 8).

Technical lead: _____ Date: _____

Security/operations: _____ Date: _____

20. Evidence Baseline

Code evaluated on branch **main** commit 709ceb9b05e7b0df939f77642e2176a8e83a9834. Key paths: models/payment_intent.py, services/processor.py, providers/mpesa.py, api/routes/payments.py, tests/test_acceptance.py, alembic/versions/001_initial_schema.py, deploy/k8s/deployment.yaml.

Open process PRs (do not change this NO-GO): #7 Engineer Mode / auth-model docs; #8 CI auto-release. Hygiene and shipping only.

Appendix principle (from gate): NetPay's core responsibility is a reliable, secure, provider-neutral representation of money movement. Provider integrations are replaceable adapters; domain, ledger, audit, idempotency, and reconciliation are the durable core. NetPay must grow that core before production release.